

Recursion - I

→ Every Recursion problem can be solved iteratively → [code might be complex]

Definition:

→ Function calling itself to solve a certain problem

Solving a problem using smaller instances of the same problem

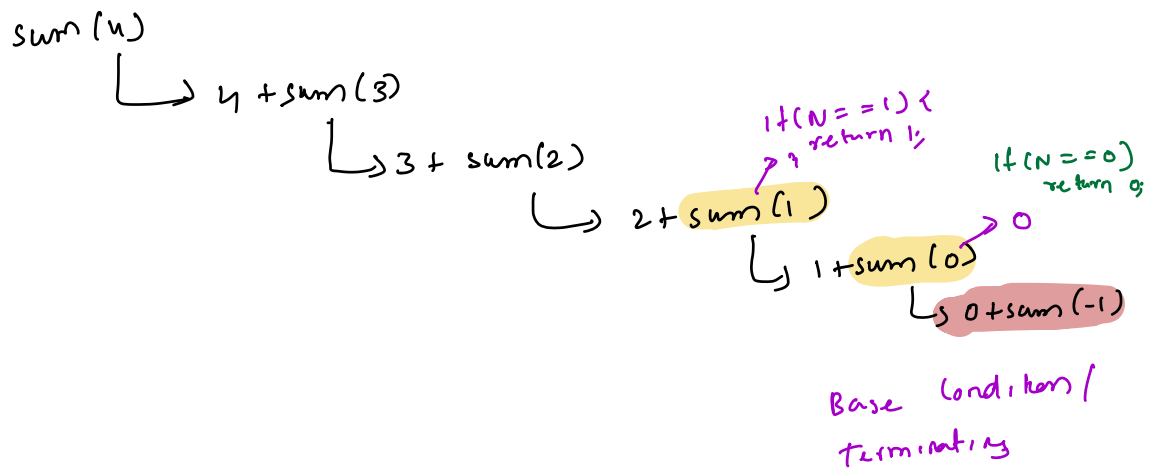
sum(N) = Returns the sum of first $N (\geq 0)$ natural numbers

$$\begin{aligned} \text{sum}(N) &= 1 + 2 + 3 + 4 + \dots + N-2 + N-1 + N \\ \text{sum}(N) &= \underbrace{1 + 2 + 3 + 4 + \dots + N-2 + N-1}_{\text{sum}(N-1)} + N \end{aligned}$$

Problem: $\text{sum}(N)$
↓
smaller instance: $\text{sum}(N-1)$
↓
Sub-problem

Solving a problem using sub-problems is called Recursion

$$\text{sum}(N) = N + \text{sum}(N-1)$$



3 steps of Recursion

1) Assumption: What do you want your function to do.
 Assume $\text{sum}(N)$ returns the sum of first N natural numbers

2) Main Logic (Recursive Relation)
 Solve the problem using smaller sub-problem

$$\text{sum}(N) = \text{sum}(N-1) + N$$

3) Base Condition
 When to stop

Option 1:

If $(N=0)$
 return 0

Option 2:

If $(N=1)$
 return 1;

$N=0$

$\text{sum}(-1) + 0$
 $\text{sum}(-2) + (-1)$

$N \geq 0$
↓

```
int sum(N) {
    if (N <= 0) {
        return 0;
    }
    return
```

$N = 0$

```
if (N == 1) {
    return 1;
}
```

↓
 $sum(-3) = -2$

```
return sum(N-1) + N;
}
```

$sum(-1) + 0$
↓
 $sum(-2) + (-1)$
↓

$$sum(N) = \boxed{sum(\frac{N}{2})} + \boxed{sum(\frac{N}{2})}$$

$$sum(6) = 1 + 2 + 3 + 4 + 5 + 6$$

$$sum(\frac{6}{2}) = \boxed{1 + 2 + 3} + 1 + 2 + 3$$

Question: Find Factorial

$$N! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot \dots \cdot (N-1) \cdot N$$

↓
fact(N-1)

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

Given N , find $N!$

3 steps

Assumption:

Assume $fact(N)$ returns $N!$

Main Logic:

$$fact(N) = fact(N-1) \times N$$

Base condition: $N == 0$
 way 1: $\{ (N == 1) \{$
 $\text{return } 1;$
 $\}$

$$0! = 1$$

$$N \geq 0$$

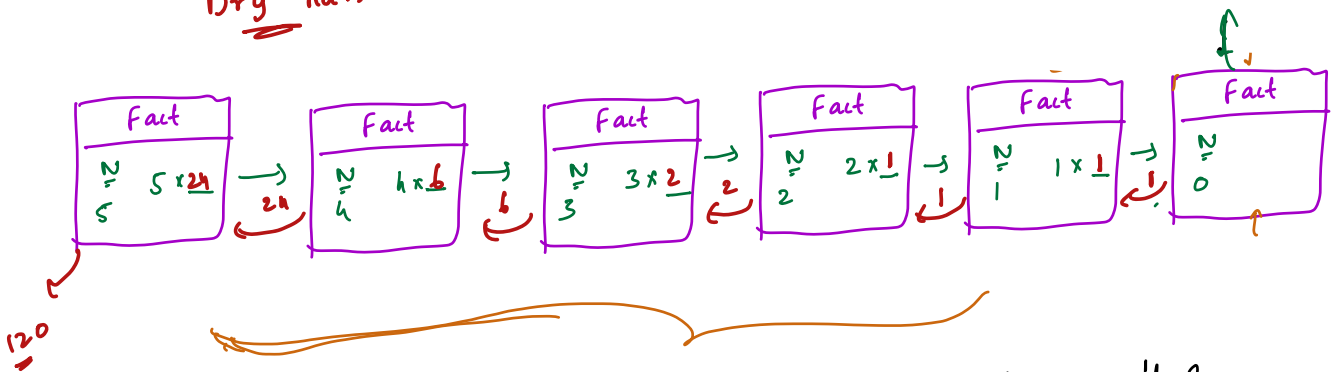
way 2: $\{ (N == 0) \{$
 $\text{return } 1;$
 $\}$

```
int fact(N) {
  if (N == 0)
    return 1;
  return fact(N-1) * N;
}
```

$\rightarrow$ $\{ (N \leq 0) \{$
 $\text{return } 1;$

$N = 2$
 $\text{fact}(2) \rightarrow 2 \times \text{fact}(1) = 2 \times 1 = 2$
 $\rightarrow \text{fact}(0) \times 1$
 $\rightarrow 1 \times 1 = 1$

Dry Run

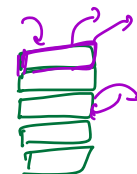


How to track all these function calls?

Stack of calls

Stack

LIFO $\Rightarrow$ Last In First Out



```

int fact(N) {
    if (N == 0)
        return 1;
    return fact(N-1) * N;
}

```

Pointers

- Whenever a function is called, memory is allocated on top of the stack.
- After the function completes execution, this is removed from the stack.

```

00000000 {
00000001 {
    int fact(N) {
        if (N == 0)
            return 1;
        return N * fact(N-1);
    }
}

```

```

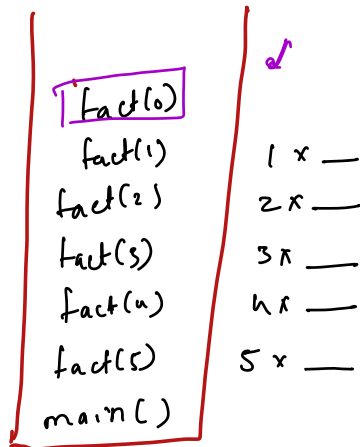
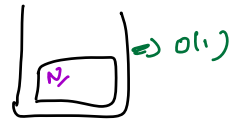
int main() {
    int ans = fact(5);
    print(ans);
    return;
}

```

```

int a, b, c;
float d;
int arr[10];

```



Stack

$$\begin{aligned}
 1 \times 1 &= 1 \\
 2 \times 1 &= 2 \\
 3 \times 2 &= 6 \\
 4 \times 6 &= 24 \\
 5 \times 24 &= 120 \\
 \text{ans} &= 120
 \end{aligned}$$

Stack

$\text{sum}(n)$

$N = n$
 $\text{sum}(n) \rightarrow \text{sum}(n-1) \rightarrow \text{sum}(n-2) \rightarrow \dots \rightarrow \text{sum}(1) \rightarrow \text{sum}(0)$
 $n+1$

Time Complexity

T.C: $O(\# \text{Function calls} \times \text{T.C per Function})$

$\text{sum}(N):$
 $\downarrow$
 $N+1$
 $\times$
 $O(1)$
 $=$
 $\boxed{O(N)}$

$\text{fact}(N) =$
 $N+1$
 $\times$
 $O(1)$
 $=$
 $O(N)$

Space Complexity $\Rightarrow$ Recursion stack

S.C: $O(\# \text{max number of active function calls} \times \text{s.c per function call})$

$\text{sum}(N) =$
 $N+1$
 $\times$
 $\boxed{O(1)}$
 $=$
 $O(N)$

$\text{fact}(N) =$
 $N+1$
 $\times$
 $O(1) \Rightarrow$
 $\boxed{O(N)}$

Iterative

T.C: $O(N)$

S.C: $O(1)$



5-10 mins

Question: Fibonacci Series

↓ ↓
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89
↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
1st 2nd 3rd 4th 5th 6th 7th 8th 9th 10th 11th
Given N, find the Nth Fibonacci number

$N = 6, 8$

$N = 9, 34$

Assumption:

fib(N) returns the Nth Fibonacci number

Main Logic:

$$\text{fib}(N) = \text{fib}(N-1) + \text{fib}(N-2)$$

Base Condition:

$$\text{if } (N == 1 \text{ || } N == 2) \text{ return } 1;$$

```
int fib(N) {
    if (N == 1 || N == 2)
        return 1;
    return fib(N-1) + fib(N-2);
}
```

$N = 2$
↓
 $\text{fib}(1) + \text{fib}(0)$
↓ ↓
1 $\text{fib}(-1) \text{ fib}(-2)$

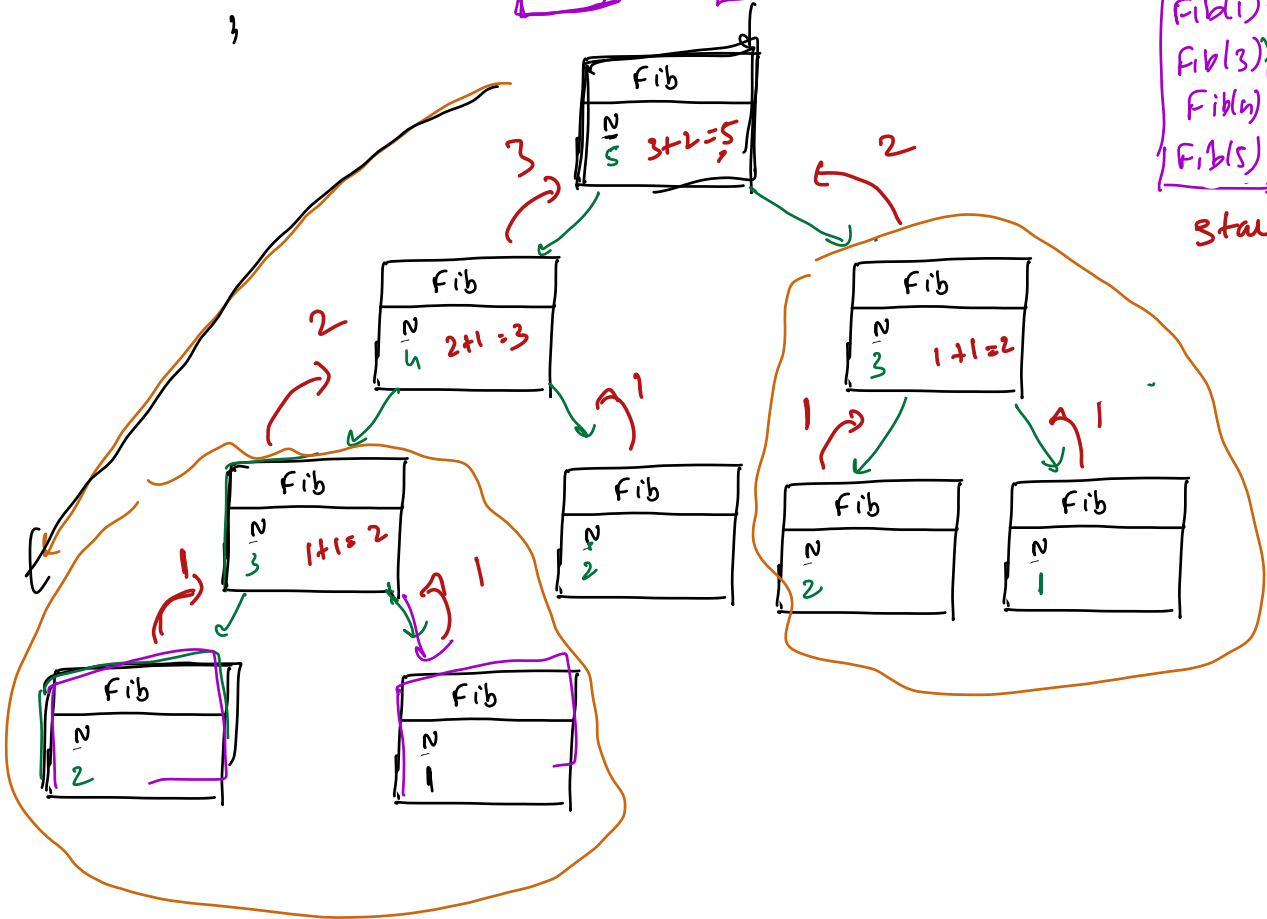
Sequentially

$O(1)$ S.C.
 $O(1)$ T.C.

```
int fib(N) {
    if (N == 1 || N == 2)
        return 1;
    return fib(N-1) + fib(N-2);
}
```

Fib(1)
Fib(3)
Fib(4)
Fib(5)

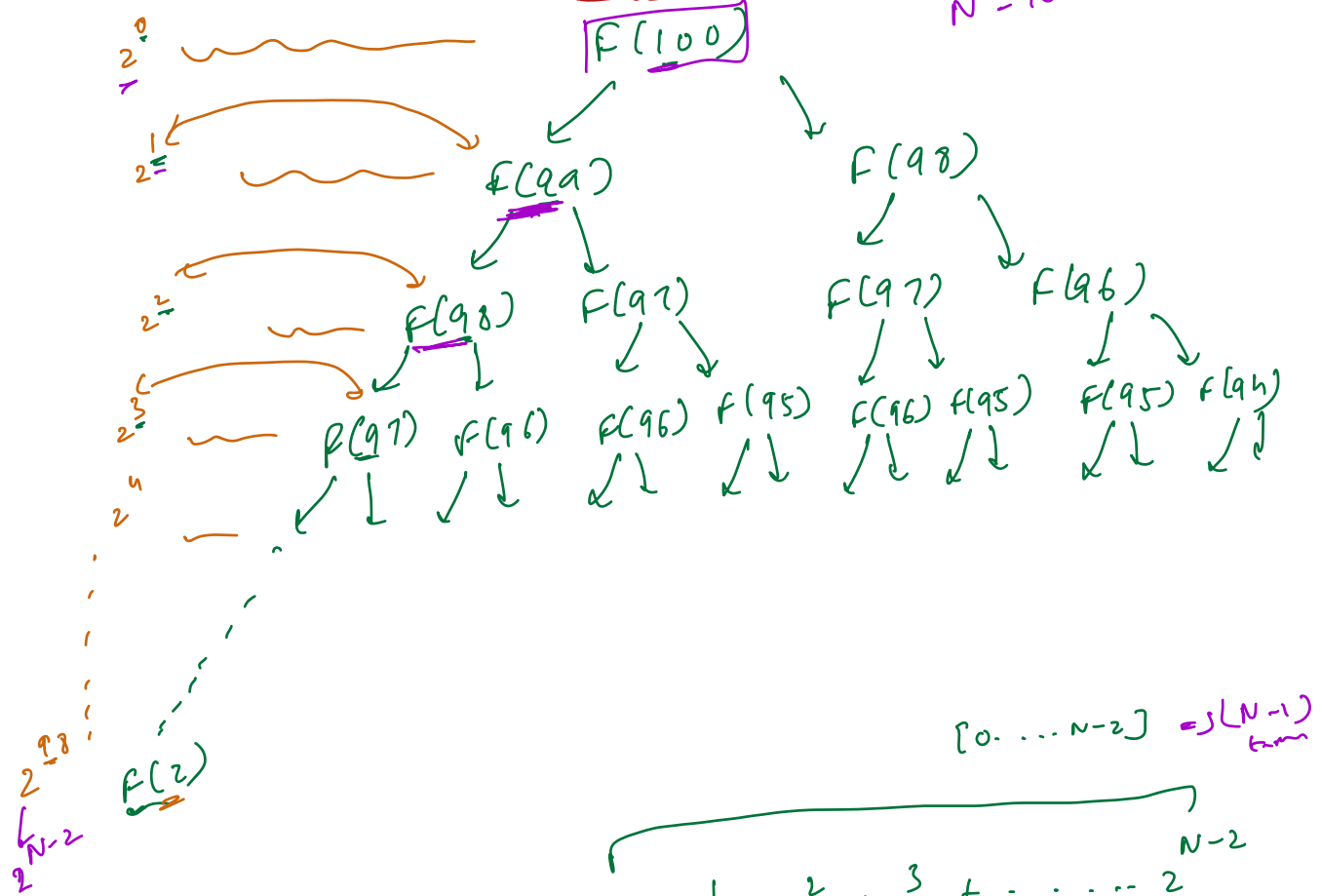
Stack



S.C.:

#Max Active Functions calls $\propto$ S.C per func
 $\downarrow$
 $N-1$
 $O(1)$

$\Rightarrow O(N)$

$$N = 100$$


function calls :

$$2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^{N-2}$$

$$\text{Sum}_k = \frac{a(r^k - 1)}{r - 1}$$

$$\begin{aligned} a &= 1 \\ r &= 2 \\ k &= N-1 \end{aligned}$$

$$= \frac{1 \binom{2^{N-1}-1}{2^{N-1}}}{2^{-1}}$$

T.C:

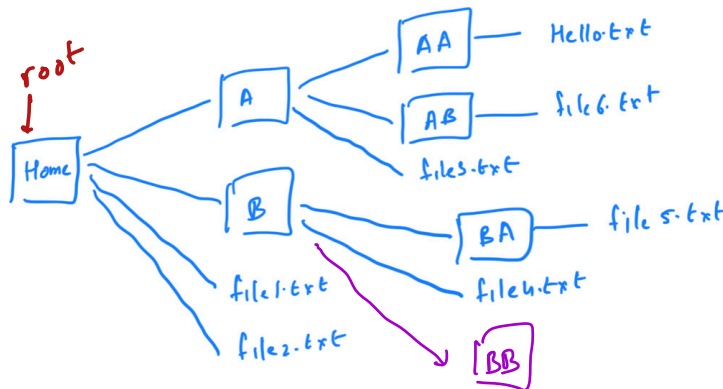
$$\frac{2^N}{2} \times O(1)$$

$$= O(2^N)$$

$$N \approx 42$$

Question: check if a file exists in a file system

isFileExist(dir, filename)
 ↓ True ↓ False



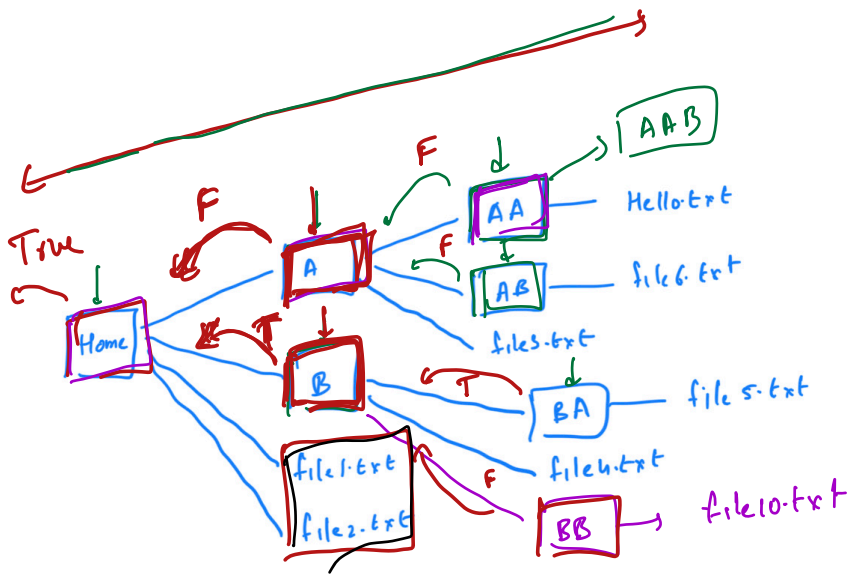
("Home", filename)

1) fetchDir(x): List all the directories inside the x.

fetchDir("Home") = ["A", "B"]
 fetchDir("A") = ["AA", "AB"]
 fetchDir("BA") = []

2) fetchFile(x): List all the files inside the x

fetchFile("Home") = ["file1.txt", "file2.txt"]
 fetchFile("BA") = ["file5.txt"]
 fetchFile("BB") = []



↑ root
↑ filename
("Home", "file5.txt")

dir → []
→ []

[
F(A)
F(A)
F(Hom)]

[]

bool isFileExists(dir, filename) {

ans

[']

O(Files in this dir)

```

if ( filename in fetchFiles( dir ) ) {
    return true;
}
// Iterate over sub-dir
for ( subdir in fetchDir( dir ) ) {
    if ( isFileExists( subdir, filename ) == true )
        return True;
}
return false;

```

D1: F
D2: F
D3: F
D4: T

```

for ( subdir in fetchDir( dir ) ) {
    isFileExists( subdir );
    F, F, T, T
}
return false;

```

0, 0, 0, 1
True

Way 1 :

T.C:

#Function calls $\times$

T.C Per Function

$\downarrow$
D
 $\downarrow$
#directory

$\downarrow$
files in that directory

$$= O(D \cdot F)$$

$\downarrow$ Max No of files in any directory

Way 2:

$$O(D + F)$$

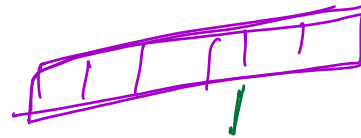
$\downarrow$ #dirs $\downarrow$ #Total No of files

S.C: (Depth of Dir Structure $\times$ Max #files in any directory)

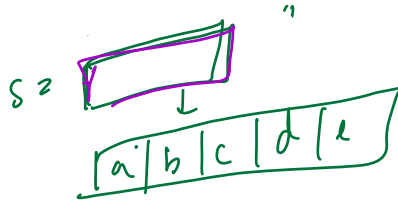
7 AM IST Mon ^{Mon} $\Rightarrow$ IST
9.30 PM EST Sun Night $\Rightarrow$ US

11 AM $\Rightarrow$

US:



character array
string builder

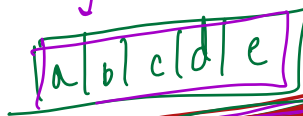


Input

str =

"a b c d e"

chars



Extra space

char array



.toString()



output

'c' 'n' 'a'

N

2^(N)

10⁷

$N \log_2$

$\log_2 10^7$

$N \approx 7 \cdot \log_2 10$

$N \approx 7 \times 3$

21